Advanced Programming 2026

Dummy Exam

This dummy exam is provided as an example of the written exam in Advanced Programming. We make no guarantees that the actual exam has the same size or level of difficulty, nor should you expect that the questions are very similar. The purpose of this dummy exam is to demonstrate how the exam questions will relate to the course material. The remaining cover text is identical to the ordinary exam, and some information (like how to hand in) are not applicable to the dummy exam.

This is the written exam in Advanced Programming. It consists of a range of questions covering the course learning goals, and you are graded based on the extent your answers demonstrate your learning. For technical reasons, your answer must be in the form of a Word document, and we have provided a template with question numbers and such prefilled for you. It is not required to use this template in case some technical obstacle prevents it.

Some of the questions are general or theoretical, but most relate to a specific assignment or weekly exercise. In those cases, any code mentioned in the question should be understood in the context the assignment or exercise in question.

1 General

1.1: Describe an advantage of representing a program's effects as data that an interpreter executes, rather than using IO directly?

Expected answer

This allows a program to be interpreted in different ways, which is particularly useful for testing, where unconstrained IO is difficult to handle.

1.2: Suppose a QuickCheck property passes 100 tests. Why might we still add a `cover` requirement to it?

> **Expected answer**
>
> A passing property says nothing by itself, since it may be 100 tests on extremely similar inputs that do not reach interesting cases. A coverage requirement ensures diversity in the test input, and makes the test fail if the input generator is too weak.

1.3: Consider this EBNF grammar for subtraction, where - is intended to be left-associative:

```
E = E "-" T
  | T
  ;

T = "1" | "2" | "3"
  ;
```

1. Rewrite the grammar to remove left recursion.
2. Sketch (in words or in parser-combinator code) how a parser for your new grammar can produce the tree `(1-2)-3` (that is, implement the left associativity properly).

> **Expected answer**
>
> 1.
> ```
> E' = "-" T E'
>    |
>    ;
> E = T E'
>   ;
>
> T = "1" | "2" | "3"
>   ;
> ```

```
pT = ...

pE = do x <- pT
        pE' x

pE' x = choice [do lString "-"
                   y <- pT
                   pE' (Minus x y),
                pure x]
```

2 Assignment questions

2.1 A1

2.1.1: Consider the code handout for A1. What do the following expressions produce?

1. `eval envEmpty (Add (Div (CstInt 1) (CstInt 0)) (Var "x"))`
2. `eval envEmpty (Add (Var "x") (Div (CstInt 1) (CstInt 0)))`
3. Are there cases where the order of evaluation of `Add` operands does not matter?

Expected answer

1. A division by zero error.
2. An unknown-variable error.
3. The order of evaluation does not matter whenever both operands terminate successfully.

2.1.2:

1. What does `ForLoop ("p", e) ("i", CstInt 0) body` evaluate to? Does it matter whether `body` fails?
2. Can this expression result in an infinite loop?

```
Let "i" (CstInt (-1))
(ForLoop ("p", e)
         ("i", Add (Var "i") (CstInt 1))
         body)
```

Expected answer

1. It evaluates to `e`, no matter what `body` does.
2. No, because the bound is evaluated with the outer binding of `i` (-1), meaning the loop never gets executed.

2.2 A2

2.2.1: One way of defining `EvalM` is this:

```
newtype EvalM a =
  EvalM (Env -> State -> (Either Error a, State))
```

Another is this:

```
newtype EvalM a =
  EvalM (Env -> State -> Either Error (a, State))
```

1. Describe what implications the difference suggests for the semantics of APL.
2. Which of these semantics most closely resemble those of conventional impure languages?

Expected answer

1. The former definition suggests that state changes are kept when an error occurs, while the latter discards state changes upon error.
2. The former resembles normal impure languages, where state changes cannot generally be rolled back.

2.2.2: Assuming a correct implementation of A2, what is the expected output of the following expression?

```
runEval (eval (Add (Print "a" (CstInt 1))
                   (Print "b" (CstInt 2))))
```

Expected answer

```
(["a: 1", "b: 2"], Right (ValInt 3))
```

2.2.3:

Consider an A2 solution where `EvalM` is defined as:

```
data State = State
  { stateOutput :: [String],
    stateKv :: [(Val, Val)]
  }

newtype EvalM a =
  EvalM (Env -> State -> (Either Error a, State))
```

And `catch` is defined as:

```
catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env state ->
  case m1 env state of
    (Left _, _) -> m2 env state
```

```
(Right x, state') -> (Right x, state')
```

1. What does the following evaluate to, and why?

```
runEval (eval (TryCatch (Div (CstInt 1)
                             (Print "a" (CstInt 0)))
                        (Print "b" (CstInt 2))))
```

2. Change `catch` so that the effects of a failing first argument remain visible to the second argument. Show the new definition of `catch`. What does the expression above evaluate to now?

Expected answer

1. `(["b:  2"], Right (ValInt 2))`, because while the division-by-zero does not happen until after the `Print`, the definition of `catch` discards the modified state.

2.
```
catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) =
  EvalM $ \env state ->
    case m1 env state of
      (Left _, state') -> m2 env state'
      (Right x, state') -> (Right x, state')
```

   Now the expression evaluates to `(["a:  0", "b:  2"], Right (ValInt 2))`

2.3 A3

2.3.1: In APL, `get` is a keyword, so `get x` is a key-value lookup, while `getx` is an ordinary variable.

1. How does the parser avoid parsing `get` as the variable name `"get"`?
2. What prevents `getx` from being parsed as the keyword `get` followed by `x`?
3. Consider parsing the input `"x y then"` as an APL expression. After `x y`, the parser tries to read another atom, reads `then`, and then fails because it is a keyword. In the APL parser, does `if x y then z else w` parse successfully? Explain why or why not.

Expected answer

1. The variable name parser `lVName` parses `get` as a name, but then rejects it by comparing against a list of known keywords.
2. The keyword parser `lKeyword` insists that a keyword is not followed by an alphanumeric character, so `getx` is rejected as a keyword.
3. Yes, this parses successfully, because `lVName` is wrapped in `try`, so the attempt to parse `then` as a variable name fails without consuming input, after which `then` can be parsed as a keyword.

2.4 A4

2.4.1: In A4, `looping m` delimits a computation, and `breakLoop v` returns `v` from the innermost enclosing `looping`.

1. What does the following evaluate to?

```
ForLoop ("p", CstInt 0) ("i", CstInt 10)
  (Apply (Lambda "x" (Break (Var "i")))
         (CstInt 0))
```

2. What does the following evaluate to?

```
Apply (Let "f" (ForLoop
                  ("p", CstInt 0) ("i", CstInt 1)
                  (Lambda "x" (Break (CstInt 7))))
           (Var "f"))
      (CstInt 0)
```

3. In terms of scoping, is "innermost enclosing loop" lexical or dynamic? How is that visible in the tree of effects?

Expected answer

1. This evaluates to `ValInt 0` because we break out of the first iteration of the loop.
2. This causes an error because the loop returns a function, and when that function is applied, it invokes a `Break` effect outside of a loop.
3. The innermost enclosing loop is a dynamically scoped notion, and this arises necessarily from the fact that the effect tree is created dynamically.

2.5 A5

2.5.1: A property such as `onlyCheckedErrors` evaluates randomly generated APL programs.

1. Can the generator in the A5 handout produce an expression for which `eval` loops forever? Give an example.
2. Is this likely to matter in practice with a uniform generator? Why?
3. Name two ways of ensuring that the test suite does not hang.
4. The APL dialect used for A5 does not have loops. Suppose the language is extended with `while`-loops as in week 7. How does this affect your answers?

Expected answer

1. Yes, by constructing a lambda that recurses infinitely, such as `(\x -> x x) (\x -> x x)`.
2. No, this is unlikely, as the generation space is very large, and these self-referential applications require a very particular structure.
3. One solution is to put an IO-based timeout on each test case. Another is to constrain the generator to avoid infinite loops, for example to make the generation type-directed rather than syntax-directed, as self-application is not well-typed.
4. It is fairly likely to produce infinite `while`-loops, as all this requires is for the condition to be tautological—and shrinking is quite likely to eventually reduce a boolean expression to plain `true`. A timeout still works, but otherwise it is tricky to avoid generation of infinite loops. One solution is to only generate `while` loops that maintain a counter, and stop the loop after some number of iterations.

2.6 A6

2.6.1: Job timeouts (as in SPC) can be enforced centrally, where the server checks deadlines as it handles messages; or locally, where each worker abandons its own job when its deadline passes. Give one advantage of each, and name an effect a client can observe.

Expected answer

The central solution requires no additional threads per job, and is arguably simpler. The downside is that we may only periodically check for timeouts, meaning detection can be delayed. Local checking requires an additional thread per job and logic in the worker for abandoning the job, and a stuck worker may not notice it. The observable effect is how long after the deadline the status becomes `JobDone DoneTimeout`.

2.7 Week 7 exercises

2.7.1: In the simulated concurrent interpreter, `step` runs a computation until the next `StepOp`, and `e1 || e2` returns the result of the computation that finishes first, in the number of steps.

1. What does `(1+2) || (3+4+5+6)` evaluate to in the simulated interpreter? Is that because the left side has fewer arithmetic operations? Can we expect always to get this result for this expression, or is it unpredictable?
2. What does `(loop p = 0 while true do p) || 1` evaluate to? Can we expect always to get this result for this expression, or is it unpredictable?

Expected answer

1. This evaluates to `ValInt 3`, because it tries the left side first. It does not matter that the left side has fewer arithmetic operations, as steps are only counted for loops. Since the simulated concurrent interpreter is deterministic, we will always get this result.

2. This evaluates to `ValInt 1`, because the left side never terminates. As before, we will always get this result.